

# EE5203 L04 Study Sheet

*Pointers, Structures, Bits, Registers, and GPIO - Basri Erdogan*

**Essential question:** How does `GPIOA->BSRR = (1U << 5);` turn on a physical LED — and how do you prove, bit by bit, that it did?

## What you should be able to do

- Distinguish a value, its address, and a pointer holding that address.
- Read `&`, `*`, `.`, and `->` in small C fragments.
- Convert between binary and hexadecimal for 32-bit registers.
- Build masks that set, clear, toggle, test, and field-replace bits.
- Decode `RCC->AHB1ENR`, `GPIOA->MODER`, `GPIOA->BSRR` and explain `volatile`.
- Enable the GPIOA clock, configure PA5 as output, and drive LD2 — then match the SFR view to the physical LED.

## Pointers in four lines

```
int x = 5;
int *p = &x;    /* p holds the ADDRESS of x      */
*p = 9;         /* write 9 THROUGH the pointer      */
               /* x is now 9; *p and x agree      */
```

| Expression                | Produces                                                    |
|---------------------------|-------------------------------------------------------------|
| <code>x</code>            | the stored value                                            |
| <code>&amp;x</code>       | the address of <code>x</code>                               |
| <code>*p</code>           | the value at the address stored in <code>p</code>           |
| <code>s.member</code>     | member of a structure value                                 |
| <code>p-&gt;member</code> | member reached through a pointer — <code>(*p).member</code> |

## Reading GPIOA->MODER

GPIOA is a CMSIS pointer to the GPIOA register structure (from the device header `stm32f401xe.h`); `->` selects a member through it; `MODER` is one 32-bit `volatile` register. `volatile` tells the compiler every read and write must really happen — hardware can change registers outside program flow.

## Mask recipes

```
value |= (1U << n);    /* set bit n      */
value &= ~(1U << n);   /* clear bit n     */
value ^= (1U << n);    /* toggle bit n    */
if (value & (1U << n)) ... /* test bit n     */

value &= ~(3U << k);   /* clear a 2-bit field at k+1:k */
value |= (1U << k);    /* then write pattern 01      */
```

Clear the whole field **before** setting the new pattern — never assume the old bits. `U` marks the constant unsigned.

## The L04 numbers

| Expression                  | Hex                     | Selects                            |
|-----------------------------|-------------------------|------------------------------------|
| <code>1U &lt;&lt; 0</code>  | <code>0x00000001</code> | GPIOA clock enable (AHB1ENR bit 0) |
| <code>3U &lt;&lt; 10</code> | <code>0x00000C00</code> | PA5 mode field, bits 11:10         |
| <code>1U &lt;&lt; 10</code> | <code>0x00000400</code> | output pattern 01 for PA5          |
| <code>1U &lt;&lt; 5</code>  | <code>0x00000020</code> | BSRR set half — PA5 high           |
| <code>1U &lt;&lt; 21</code> | <code>0x00200000</code> | BSRR reset half — PA5 low          |

- **MODER math:** pin `n` owns bits `2n+1 : 2n`; PA5  $\rightarrow$  bits 11:10; 01 = output.
- **BSRR halves:** bits 15...0 set pins; bits 31...16 reset them; reset bit = pin + 16. Each write is a command, so plain = is correct for BSRR — while MODER needs read-modify-write (`&=`, `|=`).

## The bare-metal blink, in order

```
RCC->AHB1ENR |= (1U << 0);      /* 1. clock FIRST - unlocked GPIO ignores writes */

GPIOA->MODER &= ~(3U << 10);     /* 2. clear PA5 mode field */
GPIOA->MODER |= (1U << 10);      /* 3. write 01 = output */

while (1) {
    GPIOA->BSRR = (1U << 5);      /* PA5 high - LD2 on */
    HAL_Delay(250);
    GPIOA->BSRR = (1U << 21);    /* PA5 low - LD2 off */
    HAL_Delay(250);
}
```

## Debugging register writes

Predict the bit → step the statement → read the register in the SFR view → compare hex to your mask → confirm the physical LED agrees. A successful build proves none of this.

## Common mistakes

- Using GPIOA before its clock-enable bit is set.
- Setting the new mode pattern without clearing the field first.
- A shift that selects the wrong pin (1U << 5 vs. 1U << 10 confusion — BSRR uses the pin number, MODER uses 2n).
- Confusing bitwise & | ^ with logical && || !.
- Writing = to MODER (destroys the other pins' fields).
- Treating a clean build as hardware evidence.

## Mastery self-check

- ☐ I can trace `int *p = &x; *p = 9;` and give the final value of `x`.
- ☐ I can read `p->member` as `(*p).member`.
- ☐ I can compute the hex of any `1U << n` and `3U << k` without running.
- ☐ I can write the two-statement field replace for any pin's mode.
- ☐ I can state why the RCC clock write comes first.
- ☐ I can explain the two halves of BSRR and why `plain =` is safe there.
- ☐ I can name the SFR observation that proves PA5 is an output.

## Five review questions

1. After `int a = 3; int *q = &a; *q = *q * 4;` what are `a` and `*q`?
2. Which bits does `~(3U << 6)` leave as 1, and what is the mask's purpose?
3. A student writes `GPIOA->MODER |= (1U << 10);` without the clear first. When does this still work, and what starting field value breaks it?
4. Why does driving PB0 (homework) use `1U << 0` in BSRR but bits 1:0 in MODER?
5. `volatile` is removed from the register definition and the LED stops blinking at -02. Explain what the optimizer did.

See the L04 worksheet for extended practice. Worked solutions are released after the practice window.